

InstantMessenger

Conversation Design

Project: InstantMessenger
Deadline: September 6, 2026

Members: 陈淳希 1820242104, 余善炜 1820242133, 陈瑞玲 1820242113, 谢粮璟 1820242151, 刘颖妮 1820242126

1. Conversation Definition

A conversation is an interactive text-exchange session between two or more users.

Each conversation has a unique `conversation_id` and contains the messages exchanged between its participants.

Example:

```
Conversation 10

Participants:
- Alice
- Bob

Alice: Hello Bob!
Bob: Hi Alice!
```

A conversation may also contain multiple participants:

```
Conversation 20
```

```
Participants:
```

- Alice
- Bob
- Charlie

2. Users and Participants

Each user has:

- `user_id` — unique identifier
- `username` — unique username
- `online` — current connection status

Example:

```
{  
  "user_id": 1,  
  "username": "Alice",  
  "online": true  
}
```

A conversation contains a list of participants. Only participants of a conversation can access its messages or send messages to that conversation.

When creating a conversation, the client specifies participants using their user IDs through the `participant_ids` field. Existing conversations represent these users as participants.

3. Conversation Structure

A conversation consists of:

```
Conversation
├── conversation_id
├── participants
└── messages
    ├── message_id
    ├── sender_id
    ├── sender_username
    ├── content
    └── timestamp
```

Example:

```
{
  "conversation_id": 10,
  "participants": [1, 2],
  "messages": [
    {
      "message_id": 57,
      "sender_id": 1,
      "sender_username": "Alice",
      "content": "Hello Bob!",
      "timestamp": "2026-09-01T15:30:00"
    }
  ]
}
```

4. Messages

Every message belongs to exactly one conversation.

A message contains:

- message_id
- conversation_id
- sender_id
- sender_username
- content
- timestamp

The server generates the message ID and timestamp.

Messages are displayed in chronological order.

5. Instant Messaging

When a user sends a message, the server immediately delivers it to the connected participants of the conversation.

Example:

```
Alice
|
| "Hello Bob!"
v
Server
|
v
Bob
```

If a participant is offline, the message remains in the conversation history and can be retrieved when the user reconnects.

6. Conversation Separation

Messages belonging to different conversations must remain separate.

For example:

```
Conversation: Alice + Bob

Alice: Hello!
Bob: Hi!
```

```
Conversation: Alice + Charlie
```

```
Alice: Are you free?
```

```
Charlie: Yes!
```

A message from one conversation must not appear in another conversation.

7. Conversation Lifecycle

A conversation follows this general lifecycle:

```
Create Conversation
      |
      v
Add Participants
      |
      v
Exchange Messages
      |
      v
Store Message History
      |
      v
Retrieve History Later
```

The conversation continues to exist even when all participants are temporarily offline.

8. Design Principles

The conversation design follows these principles:

1. Every conversation has a unique ID.
2. A conversation contains two or more participants.
3. Every message belongs to one conversation.

4. Only participants can access the conversation.
5. Messages are stored as conversation history.
6. Connected users receive messages immediately.
7. Offline users can retrieve previous messages later.
8. Messages from different conversations remain separated.

9. Relationship to Client-Server Protocol

The conversation design defines the concepts and behavior of the messaging system. These concepts are implemented through the client-server protocol.

For example:

- Users are managed through `login`, `get_users`, and `user_status`.
- Conversations are managed through `get_conversations` and `create_conversation`.
- Messages are retrieved through `get_messages`.
- New messages are sent using `send_message` and delivered using `new_message`.

For the detailed message formats and communication flow, see [Client-Server Protocol](#).

Client-Server Protocol

This protocol implements the concepts defined in the Conversation Design.

The Conversation Design describes the structure and behavior of the messaging system, while this document defines the messages exchanged between the client and server to implement that design.

1. Communication Method

The client and server communicate using WebSockets.

During development, the server runs at:

```
ws://localhost:8765
```

Messages exchanged between the client and server use JSON.

Every protocol message contains a `type` field.

General format:

```
{  
  "type": "<message_type>"  
}
```

2. Protocol Messages

Message Type	Direction	Purpose
connected	S → C	Indicates that the WebSocket connection has been established.
login	C → S	Allows a client to log in.
login_response	S → C	Returns the result of a login request.
get_users	C → S	Requests the list of users.
user_list	S → C	Returns the list of users.
user_status	S → C	Notifies clients when a user's online status changes.
get_conversations	C → S	Requests the conversations available to the user.
conversation_list	S → C	Returns the user's conversations.
create_conversation	C → S	Creates a new conversation.
create_conversation_response	S → C	Returns the result of conversation creation.
get_messages	C → S	Requests message history for a conversation.
message_history	S → C	Returns messages belonging to a conversation.
send_message	C → S	Sends a new message to a conversation.
new_message	S → C	Delivers a newly sent message to participants.
error	S → C	Reports an error to the client.

3. Connection

When a client connects to the server, the server sends:

```
{  
  "type": "connected",
```

```
    "protocol_version": "1.0"  
  }
```

4. Login

The client sends:

```
{  
  "type": "login",  
  "username": "Alice"  
}
```

The server responds with:

```
{  
  "type": "login_response",  
  "success": true,  
  "user": {  
    "user_id": 1,  
    "username": "Alice",  
    "online": true  
  }  
}
```

If login fails:

```
{  
  "type": "login_response",  
  "success": false,  
  "message": "Username already exists."  
}
```

5. Get Users

The client requests the current users:

```
{
  "type": "get_users"
}
```

The server responds:

```
{
  "type": "user_list",
  "users": [
    {
      "user_id": 1,
      "username": "Alice",
      "online": true
    },
    {
      "user_id": 2,
      "username": "Bob",
      "online": false
    }
  ]
}
```

6. User Status

When a user's connection status changes, the server sends:

```
{
  "type": "user_status",
  "user_id": 2,
  "username": "Bob",
  "online": true
}
```

7. Get Conversations

The client requests its conversations:

```
{
  "type": "get_conversations"
}
```

The server responds:

```
{
  "type": "conversation_list",
  "conversations": [
    {
      "conversation_id": 10,
      "participants": [1, 2]
    }
  ]
}
```

8. Create Conversation

The client requests a new conversation:

```
{
  "type": "create_conversation",
  "name": "Alice and Bob",
  "participants_ids": [1, 2]
}
```

The server responds:

```
{
  "type": "create_conversation_response",
  "success": true,
  "conversation": {
    "conversation_id": 10,
    "name": "Alice and Bob",
  }
}
```

```
    "participants": [1, 2]
  }
}
```

9. Get Message History

The client requests the messages of a conversation:

```
{
  "type": "get_messages",
  "conversation_id": 10
}
```

The server responds:

```
{
  "type": "message_history",
  "conversation_id": 10,
  "messages": [
    {
      "message_id": 57,
      "conversation_id": 10,
      "sender_id": 1,
      "sender_username": "Alice",
      "content": "Hello Bob!",
      "timestamp": "2026-09-01T15:30:00"
    }
  ]
}
```

10. Send Message

The client sends:

```
{
  "type": "send_message",
```

```
"conversation_id": 10,  
"content": "Hello Bob!"  
}
```

The server creates the message ID and timestamp.

The server then sends the message to connected participants:

```
{  
  "type": "new_message",  
  "message_id": 57,  
  "conversation_id": 10,  
  "sender_id": 1,  
  "sender_username": "Alice",  
  "content": "Hello Bob!",  
  "timestamp": "2026-09-01T15:30:00"  
}
```

11. Error Handling

When an invalid request is received, the server sends:

```
{  
  "type": "error",  
  "code": "INVALID_REQUEST",  
  "message": "Invalid request."  
}
```

Examples of errors include:

- Invalid JSON
- Missing required fields
- User not logged in
- Conversation does not exist
- User is not a participant

- Invalid message content

12. Example Communication Flow

A basic conversation can follow this sequence:



13. Client State

The client may maintain:

- Connection status
- Logged-in user
- Online users
- Available conversations
- Current conversation
- Messages for the current conversation

14. Server State

The server maintains:

- Connected users
- User information
- Conversations
- Conversation participants
- Message history
- Message IDs